

Description

Ce script a été conçu pour permettre la visualisation en continu des températures du four (et à terme des bobines) de l'expérience Sodium de l'équipe BEC du Laboratoire de Physique des Lasers (Université Sorbonne Paris Nord). Les températures sont mesurées à l'aide de thermocouples et récupérées grâce à un data logger PicoTech TC-08. L'architecture du projet est :

Driver python -> InfluxDB database -> grafana pdc agent -> grafana Cloud

Le driver python permet d'interagir avec le data logger et de récupérer les données de températures puis les exporter vers InfluxDB. Il repose sur le driver PicoSDK fourni par PicoTech sur leur site ainsi que sur la librairie influxdb3-python qui permet de communiquer avec InfluxDB. Le programme python est composé de deux fichiers : TC08_driver.py qui est une réécriture du driver natif PicoSDK afin de faciliter son utilisation, et main.py qui permet de récupérer et exporter les données. Ces fichiers sont exécutés dans un environnement virtuel dédié. Ces données sont ensuite envoyées dans une base de données InfluxDB (très efficace en terme de stockage pour des données temporelles) et sont conservées localement. L'agent pdc (Private Datasource Connection) de grafana permet à grafana Cloud de lire les données de la base et de les afficher, ce qui permet à quiconque connecté sur le cloud de voir en direct l'évolution des températures.

Utilisation en pratique

Le programme a été conçu pour qu'il puisse tourner en continu sans intervention pendant de grandes périodes de temps, notamment grâce à un fichier batch qui permet au programme de redémarrer au démarrage de l'ordinateur (fonctionne seulement sur Windows). L'agent grafana et InfluxDB qui ne nécessite aucune maintenance sont exécutés dans des terminaux dédiés en tant que sub-process (et ne peuvent donc pas être gérés manuellement). Le script Python, quant à lui, est géré dans un terminal à part qui permet une gestion manuelle, notamment si l'on veut modifier certains paramètres. En effet il peut être arrêté en appuyant Ctrl C dans le terminal dédié puis redémarré en appuyant :

```
python main.py
```

Les paramètres importants du script sont modifiables dans le fichier main.py et sont les suivants :

- **ENABLED_CHANNELS** : liste indiquant quelles channels du data logger sont utilisés
- **THERMOCOUPLE_TYPES** : dictionnaire indiquant le type des thermocouples associées à chaque channel
- **CHANNEL_LABELS** : dictionnaire des labels associés à chaque channel, sert à légender le graphique dans grafana
- **WAITING_TIME** : temps attendu par le programme entre 2 mesures, il est important de préciser que ce temps ne prend pas en compte le temps nécessaire au programme afin de récupérer les mesures et les exporter vers InfluxDB (ce qui prend à peu près 1s d'après mes tests) => le temps réel entre deux mesures est donc $\sim \text{WAITING_TIME} + 1\text{s}$
- paramètres de logs (loglevel, logfilename, logfilemode)

Ces deux derniers points peuvent être directement modifiés à l'exécution du script depuis le terminal en ajoutant les flags correspondant, pour plus d'informations tapez dans le terminal dédié :

```
python main.py --help
```

Les valeurs par défaut sont modifiables dans le script. Les logs ainsi que les erreurs seront consignés (sauf modification) dans un fichier dédié, tous comme les logs de InfluxDB et de l'agent grafana PDC.

Il suffit alors de se connecter à grafana Cloud et d'aller voir le dashboard correspondant.

N.B. : Pour des raisons qui me sont obscures, la première mesure de température pour chaque channel après exécution du script échoue systématiquement.

Affichage des données avec grafana

Pour comprendre comment afficher les données avec grafana, il faut d'abord comprendre comment elles sont stockées avec InfluxDB. Les données exportées vers influx sont stockées dans des bases de données, définies au démarrage. Au sein d'une base de données, les données vont être caractérisées par :

- une table
- des champs (dans notre cas la température)
- des tags (dans notre cas le channel et le label associé)
- un timestamp (dans notre cas l'instant de la prise de mesure)

Une mesure de température va donc être écrite de la forme suivante sous Python :

```
Point("temperatures").tag("channel",chan).tag("label",label[chan]).field("temp",temp).time(datetime.now(timezone.utc))
```

où `temperatures` est le nom de la table et `datetime.now(timezone.utc)` permet d'enregistrer la date exacte de la mesure avec le bon fuseau horaire.

L'affichage dans grafana se fait dans un dashboard. Créez un nouveau dashboard, add new elements, panel et sélectionnez Time Series dans l'onglet All visualisations sur la droite de l'écran comme type de graphe. Si lors de la connexion à votre datasource vous l'avez sélectionnée comme Make default, elle devrait déjà être renseignée, sinon sélectionnez la bonne datasource influxdb. Il ne vous reste plus qu'à sélectionner les données que vous voulez afficher sous la forme d'une requête SQL. Grafana se charge d'afficher les bonnes données, la requête sert principalement pour faire de la sélection. Par exemple :

```
SELECT temp, time, channel from temperatures  
WHERE channel >= 1 and channel <= 8
```

Cliquez sur Run query et vous devrez voir des valeurs s'afficher. Par défaut, grafana ne distingue pas les set de value et relie juste toutes les valeurs les unes aux autres. Pour les distinguer, il faut ajouter une transformation de type Partition by values en indiquant un tag de partitionnage (label ou channel) pour distinguer les différents set de données.

Pour plus d'informations, regardez la documentation de Grafana.

Démarrage manuel

En cas d'erreur dans la chaîne, le programme peut être relancé en exécutant de nouveau le fichier batch (il s'occupe de tuer influxdb et grafana pdc s'ils sont actifs).

Pour un démarrage manuel complet, vous devez :

- ouvrir un premier terminal et exécuter la commande suivante pour démarrer influxDB :

```
influxdb3 serve --node-id node0 --data-dir C:\path\to\data_logger\
```

- déplacez vous dans repertoire data_logger puis copiez collez la commande d'activation de l'agent du fichier batch, qui doit être de la forme

```
.\grafana-pdc\pdc -token GCLLOUD_PDC_SIGNING_TOKEN \  
-cluster prod-eu-west-2 \  
-gcloud-hosted-grafana-id 1703718
```

- ouvrir un troisième terminal, replacez vous dans le repertoire data_logger, activez l'environnement virtuel puis lancez le programme :

```
.\env\Scripts\activate.bat  
python main.py
```

Le programme est alors lancé et il n'y a plus qu'à se connecter à grafana Cloud pour aller voir le dashboard.

Installation complète

Les étapes ci-dessus requiert d'avoir déjà installé et configuré l'agent pdc de grafana, InfluxDB et l'environnement virtuel du fichier python. Si ce n'est pas le cas, voici comment faire. Commencez par créer un dossier data_logger dans votre arborescence :

```
mkdir C:\path\to\data_logger\
```

Ce dossier sera considéré comme la racine du projet et toute l'installation suivante est faite pour être quasi-indépendante de l'arborescence du système (bref que tout soit au même endroit).

Installation de InfluxDB3 Core

Téléchargez le binary Windows depuis leur [site](#) (Windows (AMD64,x86_64) binary). Cela ajoutera le dossier compressé correspondant dans les téléchargements. Décompressez le dans le dossier data_logger (puis supprimez le dossier compressé). Pour faciliter le suivi de ce guide, renommez le dossier en influxdb3-core.

Il faut ensuite ajouter ce dossier dans l'environnement Path de l'ordinateur. Pour cela, tapez la commande suivante dans un terminal :

```
setx Path "%Path%;C:\path\to\data_logger\influxdb3-core"
```

Cette commande permet d'ajouter le dossier influxdb3-core dans l'environnement Path de l'ordinateur, ce qui permet à Windows de reconnaître les fichiers exécutables contenues dans ce dossier comme commande. ATTENTION, cette modification ne sera pas prise en compte pour les terminal déjà ouvert mais seulement pour les nouveaux. Vous pouvez ensuite vérifier l'installation en tapant dans un nouveau terminal :

```
influxdb3 --version
```

Il devrait afficher cela :

```
influxdb3 InfluxDB 3 Core, 3.10.0, revision ...
```

Si ce n'est pas le cas, allez voir la documentation sur leur site <https://docs.influxdata.com/influxdb3/core/>. Il faut ensuite créer le token admin qui permettra de se connecter à la base de données. Pour cela il faut commencer par lancer influxdb. Pour cela tapez :

```
influxdb3 serve --node-id node0 --data-dir C:\path\to\data_logger\
```

Vous devriez voir un ensemble de log finissant par :

```
INFO : influxdb3_server: startup time: 104ms address=0.0.0.0:8181
```

Retenez le port d'écoute de InfluxDB (par défaut 8181) pour la connexion avec grafana. InfluxDB nécessite d'avoir un terminal pour lui tout seul (dans lequel il affiche ses log), ouvrez donc un autre terminal pour générer les token administrateurs en tapant :

```
influxdb3 create token --admin
```

Influxdb vous affichera votre token, conservez le et copier collez le dans la variable TOKEN de main.py. Si vous perdez ce token, il faudra le regenerer (sans quoi vous ne pourrez plus accéder à vos données)

Il faut ensuite créer la database où vont être stocké les données. Pour cela, tapez :

```
influxdb3 create database --token ADMIN_TOKEN DATABASE_NAME
```

où ADMIN_TOKEN devra être remplacé par le token que vous venez de generer et DATABASE_NAME par le nom de votre database (par exemple tempDB). Attention, le nom de votre database ne doit contenir que des lettres (a-z,A-Z), des chiffres (0-9) et _ ou -. De plus, même si ce n'est pas indiqué dans la documentation, j'ai obtenu des erreurs lorsque le nom de ma database commençait par une majuscule.

Vous pouvez maintenant utiliser InfluxDB !

Configuration du programme Python

Commencez par télécharger le driver Python du data logger sur la page de [Picotech](#) (selectionner PicoLog Data loggers puis PicoLog TC-08) et selectionnez la version de PicoSDK correspondant à votre architecture (version Windows x64 pour une machine Windows 64bit). Attention à bien choisir une version PicoSDK, les logiciels Picolog correspondent aux logiciels natifs de Picotech et non aux driver. Après avoir rempli un formulaire vous pourrez telecharger l'installateur, qui vous permettra d'installer le driver au path suivant :

```
C:\Program Files\Pico Technology
```

Pour le programme Python, copiez les fichiers correspondant depuis votre ancien ordinateur (ou recuperez le code originel depuis mon [github](#)) et placez les dans le dossier data_logger. Ouvrez un terminal et déplacez vous dans ce directory

```
cd C:/path/to/data_logger
```

Creez un dossier pour stocker les logs des différents processus avec

```
mkdir logs
```

Ensuite creez l'environnement python virtuel en tapant :

```
.\influxdb3-core\python\python.exe -m venv env
```

Cela permet de créer un environnement virtuel avec la version de python installé avec influxdb (j'ai aucune idée de pourquoi influx réinstalle son propre python, mais quitte à ce qu'il soit là autant l'utiliser pour encapsuler le projet du mieux possible). Vous pouvez ensuite activer l'environnement en tapant :

```
.\env\Scripts\activate
```

Si vous obtenez un message d'erreur due à une erreur de sécurité, il faut modifier un paramètre de sécurité de Windows. Pour cela, tapez dans le terminal :

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -scope CurrentUser
```

ce qui permet à l'utilisateur de la session d'exécuter des fichiers non signés (dont l'auteur n'est pas vérifié par Windows) qui n'ont pas été téléchargés d'Internet. Vous pouvez ensuite télécharger toutes les librairies nécessaires au script en tapant :

```
pip install -r requirements.txt
```

En cas d'erreur téléchargez à la main les librairies numpy, influxdb3-python et picosdk. Ce ne seront pas les versions utilisés à la création du code mais je ne pense pas que ça va créer de problèmes. Vous pouvez alors modifier le shebang (la toute première ligne) du fichier main.py :

```
#!C:\path\to\data_logger\env\Scripts\python
```

Normalement c'est censé permettre d'activer l'environnement virtuel env s'il n'est pas activé (bref ça évite d'avoir à l'activer manuellement) mais je n'ai pas réussi à le faire fonctionner sous Windows donc la suite n'en tiendra pas compte.

Le programme est alors prêt à l'emploi ! Vérifiez qu'il se lance bien en tapant :

```
python main.py
```

Si vous obtenez une erreur de la forme :

```
picosdk.errors.CannotOpenPicoSDKError : PicoSDK (usbtc08) not  
compatible (check 32 vs 64-bit): Could not find module 'usbtc08' (or one of
```

```
its dependencies). Try using the full path with constructor syntax
```

C'est que la librairie python n'arrive pas à trouver la bibliothèque compilé installé avec le driver. Pour cela trouvez où se situe le fichier `usbtc08.dll` sur l'ordinateur et inserez dans le fichier `C:\path\to\data_logger\env\Lib\Site-packages\picosdk\library.py` après la ligne 67 :

```
if self.name == "usbtc08"  
    library_path = r"C\Program Files\Pico  
Technology\SDK\lib\usbtc08.dll"
```

ou quelque soit le chemin absolu vers le fichier `usbtc08.dll` .

Installation de l'agent grafana PDC et connexion à grafana Cloud

Pour télécharger l'agent, allez sur le [github](#) de l'agent PDC et telechargez le binary de l'agent correspondant à votre architecture. Decomprimez le dossier dans le dossier `data_logger` et renommez ce nouveau dossier `grafana-pdc`.

Connectez vous sur votre compte Grafana Cloud et allez dans l'onglet Connections puis cliquez sur Private Data Source Connect. Cliquez sur Add private agent puis Configuration Details. Choisissez binary comme méthode d'installation puis suivez les instructions. À la fin, si Test Agent Connection affiche 1 agent connected, notez bien la commande à effectuer pour lancer l'agent grafana, qui doit être de la forme :

```
.\pdc -token GCLOUD_PDC_SIGNING_TOKEN \  
-cluster prod-eu-west-2 \  
-gcloud-hosted-grafana-id 1703718
```

Ensuite cliquez sur Create a New datasource. Vous pourrez alors selectionner InfluxDB comme type de datasource.

Pour configurer la datasource, il faut d'abord rentrer l'adresse http du port auquel InfluxDB est connecté. Grafana ne gère pas bien les adresses localhost de type `http://localhost:8181/` donc il faut remplacer localhost par l'adresse IP de la machine. Pour la determiner tapez dans un terminal :

```
ipconfig
```

et cherchez la ligne correspondant à l'adresse IPv4 :

```
Adresse IPv4. . . . . : 10.5.1.24
```

Ajoutez ensuite dans le champ URL l'adresse http complète du port sur lequel écoute InfluxDB, soit pour le port 8181 avec l'adresse IP ci-dessus :

```
http://10.5.1.24:8181/
```

Sélectionnez Influxdb Enterprise 3.x en tant que produit et SQL comme query language (vous pouvez sélectionner le langage natif de influx InfluxQL si vous êtes familier avec, mais ce n'est pas mon cas).

The screenshot shows the 'URL and authentication' configuration page in Grafana. The left sidebar lists 'Connect data source' with options: 'URL and authentication', 'Database settings', 'Private data source connect optional', and 'Save & test'. The main area is titled 'URL and authentication' and includes a text input for 'URL *' containing 'http://10.5.1.24:8181'. Below this are two dropdown menus: 'Product *' set to 'InfluxDB Enterprise 3.x' and 'Query language *' set to 'SQL'. There are also buttons for 'Advanced HTTP Settings' and 'Auth and TLS/SSL Settings'.

Vous pouvez alors remplir les database settings en entrant DATABASE_NAME et ADMIN_TOKEN dans les champs correspondants. Affichez aussi les Advanced Database Settings et cochez l'option Insecure connection.

The screenshot shows the 'Database settings' configuration page in Grafana. The left sidebar is the same as the previous screenshot. The main area is titled 'Database settings' and includes a text input for 'Database *' containing 'tempDB'. Below this is a 'Token *' section with a 'configured' status and a 'Reset' button. There is a button for 'Advanced Database Settings'. Under this, there is a 'Max series' section with a text input set to '1000'. At the bottom, the 'Insecure Connection' checkbox is checked.

Dans l'onglet Private Data Source Connect, sélectionnez l'agent que vous venez de lancer.

The screenshot shows the 'Connections > Data sources > influxdb3 core - pdc' page in Grafana Cloud. The left sidebar has a 'Connect data source' section with options: 'URL and authentication', 'Database settings', 'Private data source connect optional' (selected), and 'Save & test'. The main area shows the configuration for 'tempDB'. It includes a 'Token' field set to 'configured' with a 'Reset' button. Below is an 'Advanced Database Settings' section with a 'Max series' field set to '1000' and an 'Insecure Connection' checkbox checked. At the bottom, there is a 'Private data source connect' section with an 'optional' label. It contains a description, a 'Private data source connect network' dropdown menu showing 'pdc-cheerfulshelf2011-default (1 agent connected)', and a link to 'Manage private data source connect networks'. At the very bottom are three buttons: 'Delete', 'Remove default', and 'Save & test'.

Vous pouvez alors enregistrer votre datasource en cliquant sur Save & Test. Si vous recevez un message de succès, vous avez bien connecté votre datasource et vous pouvez aller visualiser les données dans un dashboard ! Pensez à cliquer sur Make Default si ce projet est votre utilisation principale de Grafana Cloud.

Vous avez maintenant configuré votre datasource !

Pour lancer l'agent, ouvrez un terminal et placez vous dans le directory

```
C:\path\to\data_logger\grafana-pdc
```

puis copiez collez la commande que vous donnait grafana Cloud.

Configuration du fichier batch et du redemarrage automatique

On peut automatiser le projet grâce à un fichier batch qui va permettre de démarrer InfluxDB, l'agent pdc de grafana et le programme Python ! Pour cela ouvrez le fichier run_temps_acq.bat et modifiez la ligne 4 pour y placer le path vers data_logger :

```
cd /d "C:\path\to\data_logger"
```

Modifiez la ligne 7 de tel sorte qu'elle corresponde à :

```
start "" /b ".\influxdb3-core\influxdb3.exe" serve --node-id node0 --
data-dir C:\path\to\data_logger
```

Modifiez ensuite la ligne 15 pour qu'elle corresponde à la commande utilisé pour lancer l'agent grafana

```
start "" /b ".\grafana-pdc\pdc" -token GCLLOUD_PDC_SIGNING_TOKEN -  
cluster prod-eu-west-2 -gcloud-hosted-grafana-id 1703718
```

Faites attention à rajouter le chemin vers le dossier grafana-pdc dans le path vu qu'on se situe dans le directory data_logger.

Vous ensuite tester que le batch fonctionne bien, en double-cliquant dessus il devrait ouvrir deux terminaux : un terminal auxiliaire gerant le programme python et le terminal qui a reçu les instructions du batch.

Pour que le programme redemarre automatique au démarrage de l'ordinateur, il faut ajouter le fichier batch au planificateur de tâches. Pour cela ouvrez le planificateur de tâches windows puis cliquez sur Créer une tâche. Remplissez le nom et le descriptif, puis ajoutez un déclencheur dans le menu déclencheur avec comme paramètre de lancement de tâche À l'ouverture de la session puis cliquez sur Ok. Dans le menu Actions, ajouter une nouvelle action de type Démarrer un programme et parcourez votre arborescence jusqu'à sélectionner le fichier run_temps_acq.bat. Une fois que c'est fait, validez pour confirmer et enregistrer la nouvelle tâche. Maintenant le programme devrait s'exécuter lorsque l'utilisateur sélectionné se connectera à l'ordinateur après un redémarrage !

Regeneration du token de la database

Si vous avez perdu votre token, vous pouvez le regénérer de la façon suivante : s'il est déjà lancé stoppez le processus InfluxDB puis redémarrez votre serveur avec le tag :

```
influxdb3 serve --node-id node0 --data-dir C:\path\to\data_logger\ --  
admin-token-recovery-http-bind
```

la dernière ligne de logs mentionnera alors :

```
INFO : influxdb3_server: starting admin token recovery endpoint on  
address=127.0.0.1:8182
```

Notez cette adresse, ouvrez un autre terminal et tapez :

```
influxdb3 create token --admin --regenerate --host  
http://127.0.0.1:8182
```

Cela vous permettra de régénérer votre token. Il vous faudra alors modifier le fichier python pour y inclure le bon token, et modifier aussi le token dans les paramètres de la datasource sur Grafana.

Suppression complète du programme et de ses dépendances

Pour supprimer l'entièreté des fichiers liés à ce projet, commencez par fermer tous les processus en cours. Ensuite ouvrez le planificateur des tâches et supprimez la tâche data logger (ou quelque soit le nom que vous aviez choisi). Ensuite ouvrez les propriétés systèmes pour modifier les variables d'environnement (chercher modifier les variables d'environnement dans l'outil de recherche), cliquez sur variable d'environnement, sélectionnez Path et cliquez sur modifier. Sélectionner le chemin correspondant à

```
C:\path\to\data_logger\influxdb
```

et supprimez le. Pour supprimer le driver PicoSDK, naviguez dans l'arborescence de votre ordinateur jusqu'à trouver un dossier Pico Technology situé dans le dossier Programmes ou Program Files de votre ordinateur (situé sur le disque local) et supprimez le dossier Pico Technology. Il ne vous reste alors plus qu'à supprimer le dossier :

```
C:\path\to\data_logger
```